

Functions (advanced)

Brainster Web Development Academy



Table of contents

01 Functions

02 Exercise 1

03 Arrow functions

04 Exercise 2

05 Exercise 3

06 Exercise 4

07 Exercise 5



Basic functions

As a reminder, the two ways to define a function were:

Named function

```
function functionName(parameters)
{
  // Do stuff here
}
```

Anonymous Function

```
let functionName = function(parameters)
{
  // Do stuff here
}
```



Exercise 1

1. Write a function that:
 - 1.1. Add two numbers and return the result
 - 1.2. Subtract two numbers and return the result
 - 1.3. Multiply two numbers and return the result
 - 1.4. Divide two numbers and return the result
2. Define two number variables
3. Invoke each of the four function on those two numbers
4. Log the results in the console



Arrow functions

There is a newer, third way to define a function (since 2015/ES6) called an arrow function:

```
let myArrowFunction = (parameter) => {  
  // do stuff here  
};
```

For the most part, the same rules apply as with regular functions - only these are smaller and (sometimes) more readable.



Arrow functions

They come with an extra “superpower”, and that is the optional curly braces usage:

Implicit return in arrow function

```
let add = (a, b) => a + b; // Explicit return
```

Written as a regular function, this would translate to:

Explicit return in arrow function

```
function add(a, b) {  
  return a + b; // Explicit return  
}
```



Exercise 2

1. Rewrite each of the functions from exercise one using arrow functions.
Try to use the minimum amount of lines / characters possible to achieve the result.
2. Add a function that finds the **square** of a given number (also using an **arrow function**).
3. Add a function that finds the **cube** of a given number (also using an **arrow function**).
4. Define two number variables
5. Invoke each of the function on those two numbers



Exercise 3

1. Create a function that takes an array as input and returns that **arrays length** as output.
2. Try the function on different arrays
3. Rewrite the exercise with **arrow function** in one line.



Exercise 4

1. Prompt for a word.
2. Create an arrow function that will take a word as in input and return a **reversed** version of that same word.

(**Hint:** make an array from the given word & then use array methods on it.)



Exercise 5

1. Create a function that will take an array as an input
2. Calculate and return the **sum** of all the elements of the given array.
3. Try if the same function works on this array **[5, '5', 3]** which has an element as a string.

